PHASE 1 · ENGINEERING REPORT & CODE REVIEW

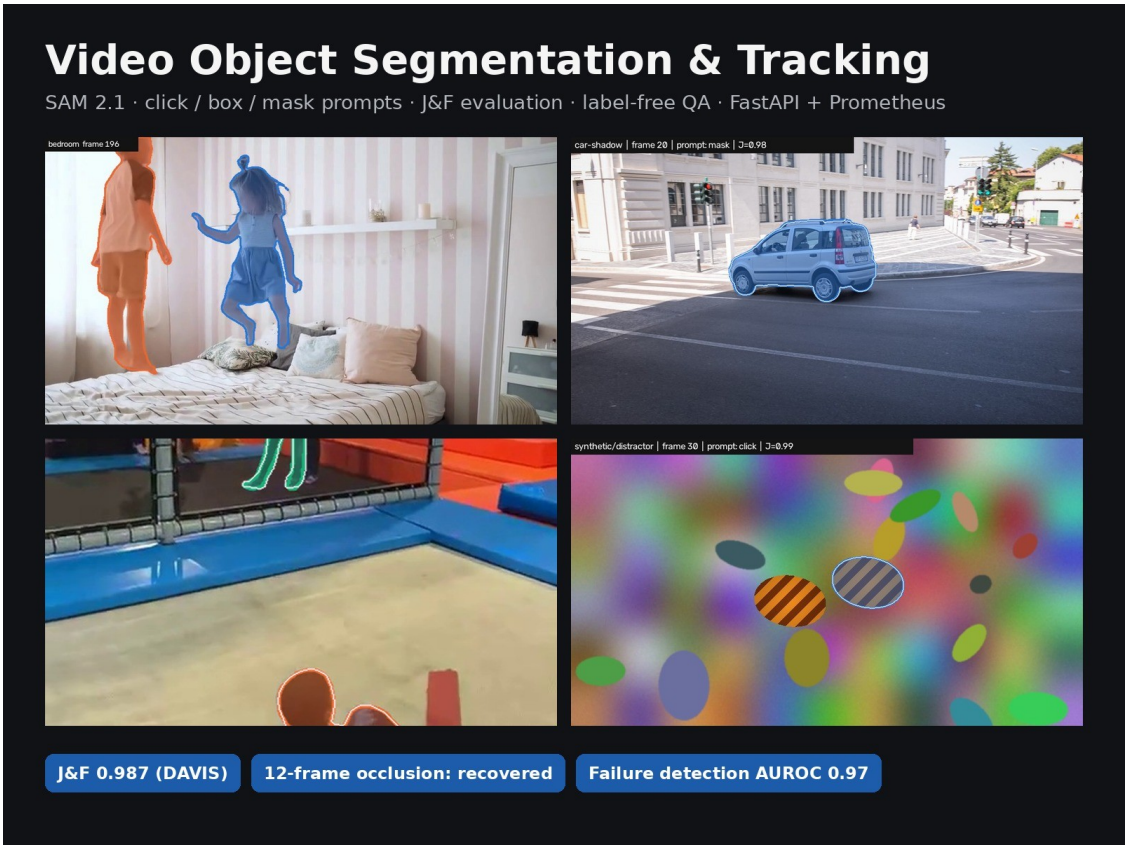
SAM 2 Video Segmentation & Tracking Platform

Can a segmentation mask be kept on an object across a whole video, and how do you prove it?

Built on: ML Model Serving Platform (FastAPI · PostgreSQL registry · Prometheus · Grafana)

Model: **sam2.1-hiera-tiny** (38,962,498 parameters) · runtime: PyTorch 2.14.1+cu130 on cpu (2 threads) · date: 2026-10-13

Author: Nirvana Fanaelahi · Repository: sam2-video-seg-platform



Contents

1 **Executive summary** · the answer, key findings, what was built

2 **Question, protocol and data** · what is measured and on which clips

3 **Results** · accuracy, persistence, failure modes, label-free signals, cost

4 **Architecture** · what changed in the base platform; file inventory

5 **Code review** · per script: purpose, technique, findings, status; key code explained

6 **Techniques explained** · learning notes: SAM 2 memory, VOS metrics, flow, caching, tracking

7 **Recommendations and roadmap** · prioritised fixes, Phases 2–5

8 **Reproducibility** · commands, environment, checksums

A **Upwork portfolio entry** · form fields and content blocks

1. Executive summary

Yes, with one specific exception. SAM 2.1 (tiny) holds a pixel-accurate mask through a whole clip (J&F 0.987 on DAVIS car-shadow, J-decay ≈ 0.01). One click is as good as a full mask on a salient object. It returns an empty mask while an object is fully occluded and re-acquires it on the first visible frame. A look-alike crossing in front does not steal the identity. The exception is an object that leaves the frame and comes back changed: SAM 2 re-detected it, lost it again for 14 frames, then recovered. That failure is visible without labels, because SAM 2's own object-presence score hovers just below zero the whole time. That is the trigger a production system needs for automatic re-prompting, which is the subject of Phase 2.

Key findings

- **Accuracy:** J&F 0.987 on DAVIS car-shadow from a full mask. Box, 3 clicks and 1 click give 0.987, 0.987 and 0.987: for a salient object, user effort is one click.
- **Persistence through occlusion:** 12 fully hidden frames → hidden-frame J = 1.00 (empty on every hidden frame), re-detection delay [0] frames, J&F 0.988.
- **Look-alike distractor:** no identity switch. The mask leaks onto the distractor for 1–2 frames at the crossing (max 64 % of the mask), J&F 0.976.
- **Failure mode found:** exit and re-entry gives J&F 0.672, J-decay 0.41. After re-detection the object was lost again for 14 frames while fully visible, with the object score just below 0 (median -0.42).
- **Fast motion with blur:** the object is never lost (J recall 0.98), but masks swell over the blur streak, so F drops to 0.56. Binary masks are the wrong output for motion blur; roto work needs soft alpha (matting).
- **Label-free monitoring works:** the SAM 2 object score flags frames with J < 0.5 at AUROC 0.97 (32 failure frames among 988 scored). It is wired into Prometheus as quality alerts.
- **Real footage:** on the SA-V example (5 objects, 6 fps) J&F is 0.944. In the bedroom clip (no GT) both children keep their identity through a mutual occlusion and partial exits at the frame edge, processing only every 4th frame.
- **Cost (2 vCPU, no GPU):** image encoder ≈ 2.3 s/frame plus ≈ 3.6 s per object per frame for memory attention. Cost grows linearly with the number of objects, which is exactly what SAM 3.1's Object Multiplex targets. Processing every 4th frame cut the encoder cost 4× with no loss on smooth motion.

What was built

- **Evaluation harness** (`src/vos`): a DAVIS-protocol port of J, F, J&F, recall and decay, with an upstream decay bug fixed. It adds event metrics for occlusion, re-entry and distractors, and GT-free stability metrics.
- **Synthetic stress tests** with exact ground truth, one per failure mode: 12-frame full occlusion, an identical distractor crossing in front, a 12-frame exit and re-entry, and 50 px/frame motion with blur.
- **SAM 2.1 wrapper** with a persistent feature cache (bit-identical masks; re-runs ≈ 1.7× faster for one object), the object-presence score exposed, and late-appearing objects handled.
- **Serving**: SAM 2 checkpoints go through the existing registry, storage, LRU cache and lifecycle. A new multipart video endpoint returns COCO-RLE masks, a stability report and quality alerts. Prometheus metrics and a Grafana dashboard cover it.
- **Rotoscoping CLI**: clicks or a box in; alpha-matte PNGs, an overlay MP4, masks and a stability report out.
- **Quality gates**: 116 fast tests (SQLite, fake segmenter) + 6 slow tests with real SAM 2 inference, Ruff lint and format checks, and a CI job with PostgreSQL.

2. Question, protocol and data

2.1 The Phase-1 question

A client asking for “AI rotoscoping”, “track this object” or “blur every face” really asks three things. Does the mask stay **accurate** on every frame? Does it **persist** through occlusions, exits and look-alikes? Will I **know** when it fails, without labelling every frame? Phase 1 answers each with a measurable quantity:

Client question	Measured as	Where
Is the mask accurate?	J (IoU) and F (boundary F-measure), J&F, DAVIS protocol	metrics.py
Does it persist over time?	J-decay (first vs. last quarter); per-frame J curves	metrics.py, report.py
Does it survive occlusion / exit?	hidden-frame J (empty is correct), re-detection delay	benchmark.py
Does it jump to a look-alike?	distractor leakage, identity-switch flag	benchmark.py
How much user effort?	J&F for mask vs. box vs. 3 clicks vs. 1 click	prompts.py
What does it cost?	ms/frame, encoder vs. per-object memory cost, stride trade-off	cli.py profile
Will I know when it fails?	AUROC of GT-free signals for detecting frames with J < 0.5	stability.py, drift_detector.py

2.2 Protocol

- **Semi-supervised VOS.** Each object is prompted once, on its first visible frame, with a prompt derived from the ground truth. Nothing else is given; SAM 2 propagates forward.
- **Scoring.** DAVIS conventions: the first (prompt) and last annotated frames are excluded; an empty prediction on an empty GT scores J = 1; dataset scores average over objects. Boundary tolerance is ⌈0.008 × image diagonal⌉ (8 px at 480p).
- **Sparse GT.** SA-V is annotated at 6 fps (every 4th frame), so it runs at stride 4 and only annotated frames are scored.
- **Determinism.** Inference is deterministic, so accuracy numbers are exact. Timings come from a shared 2-vCPU sandbox and carry about ±20 % noise.

2.3 Data

Sequence	Frames	Objects	What it tests	Caveat
DAVIS car-shadow (val)	40	1	Standard VOS benchmark clip	One easy sequence. The full DAVIS 2017 val host was not reachable from the build environment.
SA-V sav_000001	483 (121 scored)	4 visible	Real multi-object video; one object with 10 visibility changes	SA-V train split: SAM 2 was trained on it. Treat as a pipeline check.
synthetic/occlusion	48	2	12 frames fully hidden behind a pillar; never-occluded control object	Synthetic
synthetic/exit_reentry	48	1	12 frames outside the image	Synthetic
synthetic/distractor	48	1 (+1 look-alike)	Identical object crosses in front of the target	Synthetic
synthetic/fast_motion	48	1	≈50 px/frame zig-zag, blur, rotation, scale change	Synthetic
bedroom (SAM 2 repo)	200 (50 used)	2	Two children crossing; GT-free stability + demo	No ground truth

3. Results

All numbers come from `results/phase1/runs.json`, produced by `make benchmark` (config `configs/phase1.yaml`) on x86_64 with 2 threads. Checkpoint SHA-256: `3c1e81ca9b037dd3…` (torch.float16).

3.1 Overview

Sequence	Prompt	Stride	Frames	Obj.	J&F	J	F	J-decay	ms/ frame
car-shadow	mask	1	40	1	0.987	0.976	0.998	+0.013	6,136
car-shadow	box	1	40	1	0.987	0.976	0.998	+0.012	3,298
car-shadow	click	1	40	1	0.987	0.976	0.997	+0.011	3,096
car-shadow	click3	1	40	1	0.987	0.977	0.998	+0.010	3,085
car-shadow	mask	2	20	1	0.987	0.976	0.998	+0.013	5,127
car-shadow	mask	4	10	1	0.988	0.977	0.998	+0.010	2,201
synthetic/occlusion	mask	1	48	2	0.988	0.976	1.000	-0.001	9,412
synthetic/exit_reentry	mask	1	48	1	0.672	0.670	0.674	+0.412	5,529
synthetic/fast_motion	mask	1	48	1	0.640	0.718	0.561	+0.020	5,652
synthetic/distractor	mask	1	48	1	0.976	0.968	0.984	+0.001	5,680
synthetic/distractor	click	1	48	1	0.975	0.968	0.982	-0.003	3,407
sav_000001	mask	4	121	5	0.944	0.918	0.970	+0.045	16,166
synthetic/fast_motion	mask	2	24	1	0.634	0.719	0.549	+0.034	5,287
synthetic/fast_motion	mask	4	12	1	0.693	0.754	0.633	-0.007	2,561
synthetic/occlusion	mask	2	24	2	0.989	0.978	1.000	-0.002	8,217
synthetic/occlusion	mask	4	12	2	0.990	0.981	1.000	+0.011	5,107

J-decay is the mean J over the first quarter of the scored frames minus the mean over the last quarter. A value near zero means the mask does not degrade over time. The ms/frame column includes cache effects (the first run on a video computes the image encoder; later runs reuse it). See §3.6 for the clean cost model.

3.2 How much user input is needed?

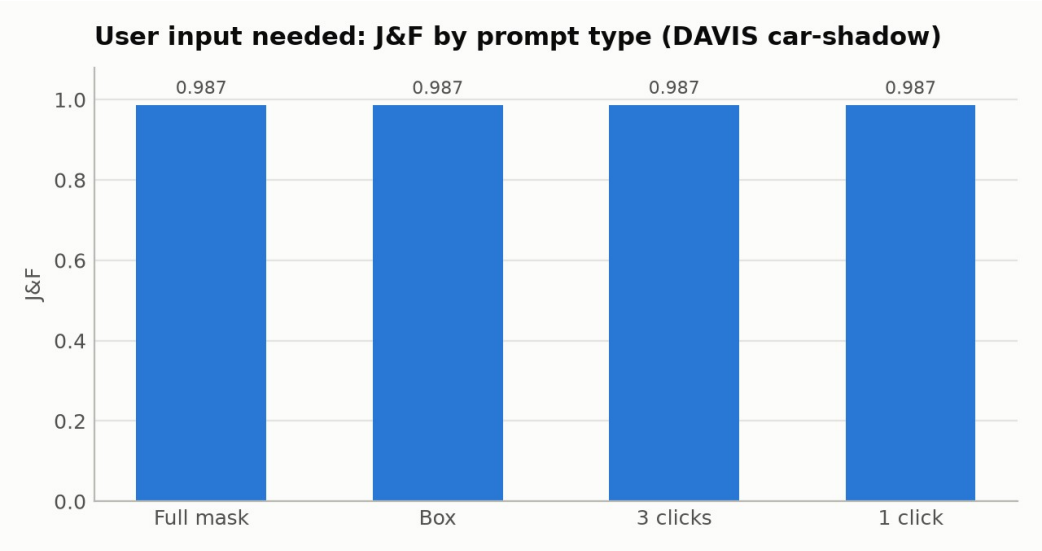


Figure 1 – J&F on DAVIS car-shadow for four prompt types (single run each, deterministic).

car-shadow is a salient, well-separated object, so the prompt type barely matters: every prompt converges to the same mask after a few frames. That is useful to a client, because one click per object is enough for this class of shot. The prompt type will matter for thin, multi-part or cluttered objects; the harness makes that a one-line config change on any DAVIS sequence.

3.3 Frame sub-sampling: accuracy vs. cost

Sequence	Stride	Effective rate	Frames	J&F	J	J-decay	Encoder cost
car-shadow	1	24 fps	40	0.987	0.976	+0.013	1.00×
car-shadow	2	12 fps	20	0.987	0.976	+0.013	0.50×
car-shadow	4	6 fps	10	0.988	0.977	+0.010	0.25×
synthetic/fast_motion	1	24 fps	48	0.640	0.718	+0.020	1.00×
synthetic/fast_motion	2	12 fps	24	0.634	0.719	+0.034	0.50×
synthetic/fast_motion	4	6 fps	12	0.693	0.754	-0.007	0.25×
synthetic/occlusion	1	24 fps	48	0.988	0.976	-0.001	1.00×
synthetic/occlusion	2	12 fps	24	0.989	0.978	-0.002	0.50×
synthetic/occlusion	4	6 fps	12	0.990	0.981	+0.011	0.25×

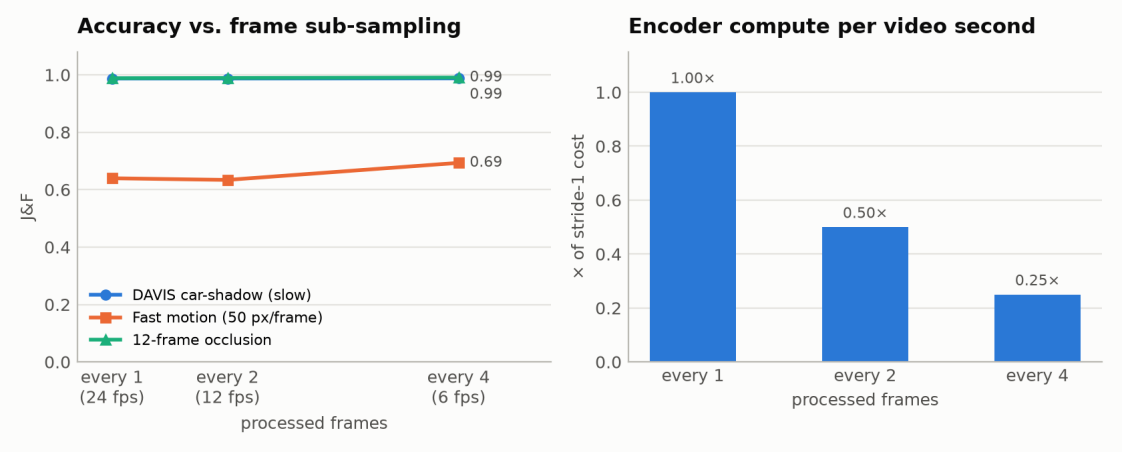


Figure 2 – Accuracy (left) and relative encoder compute (right) when only every 2nd / 4th frame is processed.

On car-shadow (slow, smooth motion), processing only every 4th frame keeps J&F at 0.988, a 4× saving on the encoder. The 12-frame occlusion is also unaffected (0.988 / 0.989 / 0.990 at stride 1 / 2 / 4): with 6 hidden frames instead of 12, re-detection is still immediate. The fast-motion clip stays at 0.640 / 0.634 / 0.693. Its error comes from the mask covering the motion-blur streak, not from losing the object, so skipping frames neither helps nor hurts much. Rule of thumb for quoting a job: sub-sample when the displacement between processed frames stays well below the object size. Measure it on a sample clip with this harness before promising a frame rate.

3.4 Failure modes: does the mask persist?

Scenario	J&F (object)	Hidden frames	Hidden-J	Re-detect delay (frames)	J after 1st hide	Distractor leakage	ID switch
synthetic/occlusion	0.990	12	1.000	0	0.971	–	–
synthetic/exit_reentry	0.672	12	1.000	1	0.372	–	–
synthetic/distractor	0.976	0	–	–	–	2.3 %	no
synthetic/distractor (click)	0.975	0	–	–	–	1.5 %	no

Scenario	J&F (object)	Hidden frames	Hidden-J	Re-detect delay (frames)	J after 1st hide	Distractor leakage	ID switch
synthetic/fast_motion	0.640	0	–	–	–	–	–
sav_000001 obj 1	0.872	1	1.000	0	0.823	–	–
sav_000001 obj 3	0.912	11	0.818	0, 0, 0, 0, 0	0.872	–	–
sav_000001 obj 4	0.997	113	1.000	0	0.901	–	–

Hidden-J is the J over frames where the object is fully invisible; 1.0 means SAM 2 correctly returned an empty mask on every one of them. Re-detection delay counts the frames after reappearance until J ≥ 0.5 (0 means the first visible frame). Distractor leakage is the share of the predicted mask that lies on the look-alike object.

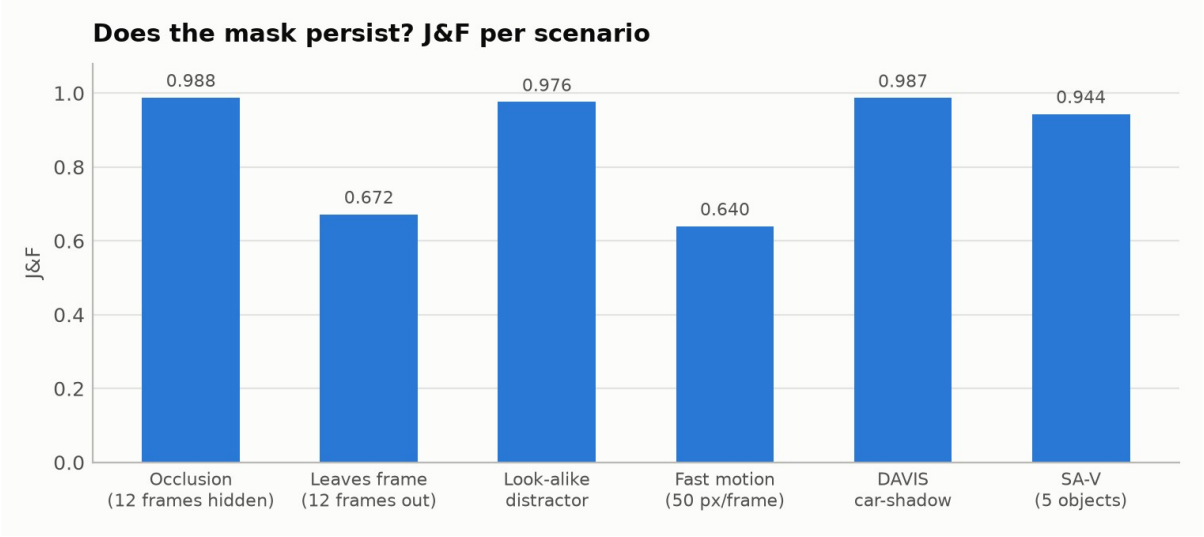


Figure 3 – J&F per scenario (mask prompt on the first frame).



Figure 4 – Per-frame J. Grey bands are frames where the object is fully hidden; the correct output there is an empty mask (J = 1).

Occlusion. SAM 2's object-presence score went negative exactly on the hidden frames (≈ −3.6 to −5.2) and the mask was empty on all 12 of them. The control object, never occluded, stayed at J ≈ 0.99. This is the memory

bank working as designed: the prompted frame stays in memory, so the object is recognised again even after it vanished.

Exit and re-entry: the instructive failure. The target left the frame for 12 frames and came back rotated. SAM 2 caught it on the second visible frame (J 0.97), then lost it again for 14 frames while it was fully visible. Its object score stayed between −1.2 and 0 until it crossed zero and tracking resumed. The memory window was full of "absent" frames and the rotated appearance only weakly matched the prompt. This is precisely what SAMURAI (motion-aware memory selection) and DAM4SAM (distractor-aware memory) address. A cheap production fix is a hybrid: when the object score sits between about −2 and 0.5 for several frames, re-prompt SAM 2 from a detector box.

Look-alike distractor. An identical object crossing in front pulled up to 64 % of the mask onto itself for one or two frames, but no identity switch occurred and J was back at 0.98 one frame later. With a single click the result is the same (J&F 0.975, max leakage 32 %).

Real clip without labels (bedroom). Two children, two clicks each, processed at 6 fps. Both keep their identity when the girl passes in front of the boy and when the boy is cut off at the left edge of the frame. The object score never goes negative (min 3.3). Temporal IoU is low (≈0.4–0.5) only because limbs move a lot between frames 4 apart at 24 fps; flow-warped IoU is higher, but optical flow itself is unreliable at that displacement.

Fast motion. At ≈ 50 px/frame with motion blur the object is never lost, but the mask covers the blur streak while the ground truth is the sharp silhouette. Both answers are defensible, so for VFX the right deliverable is a soft alpha matte (SAM 2 mask → matting model), not a binary mask.

3.5 Real multi-object video (SA-V example)

Object	Frames scored	J&F	J mean	J recall	J-decay	Hidden frames	Reappearances	Lost again
1	119	0.872	0.831	0.908	+0.223	1	1	11
2	119	0.971	0.958	1.000	-0.001	0	0	0
3	119	0.912	0.863	0.966	-0.030	11	5	2
4	119	0.997	0.994	1.000	+0.024	113	1	0
5	119	0.967	0.942	1.000	+0.009	0	0	0

Mean J&F 0.944 over 5 objects at 6 fps. The object with 10 visibility changes (#3) was re-detected on the first visible frame at each of its 5 reappearances, but SAM 2 predicted a mask on 2 of its 11 hidden frames (hidden-J 0.82). The smallest object (#1, 0.35 % of the image) is the only one that degrades over time (J-decay 0.22). Small objects are where memory drift shows first. Object #4 appears only in the last 8 annotated frames; it was tracked in a separate pass from its first appearance, and its high score is dominated by correctly empty frames (the same convention as DAVIS). This video is from SA-V train, so it is a pipeline check, not a generalisation claim. Peak RAM was 5.5 GB for 121 frames, which is why the API caps frames per request.

3.6 GT-free quality signals and the cost model

Signal (no ground truth needed)	Spearman ρ with J	AUROC: detect J < 0.5
SAM 2 object-presence logit	0.47	0.97
Mean foreground probability in mask	0.86	0.94
Share of uncertain pixels (0.3 < p < 0.7)	-0.03	0.18

Computed over 988 visible-object frames, of which 32 have J < 0.5. SAM 2's object-presence logit is the best single signal (AUROC 0.97): it goes negative when the model believes the object is gone, so it catches both correct "hidden" frames and the exit-and-re-entry failure. Mask confidence also separates good from bad frames (AUROC 0.94), but it is only defined when the mask is non-empty (13 of 32 failure frames), so it cannot see an object that was dropped. The uncertain-pixel share is inverted (AUROC 0.18): a lost object produces a

confidently empty mask with no uncertain pixels. So it is not a failure detector on its own. The production monitor therefore combines object score, temporal IoU and drop-out counts.

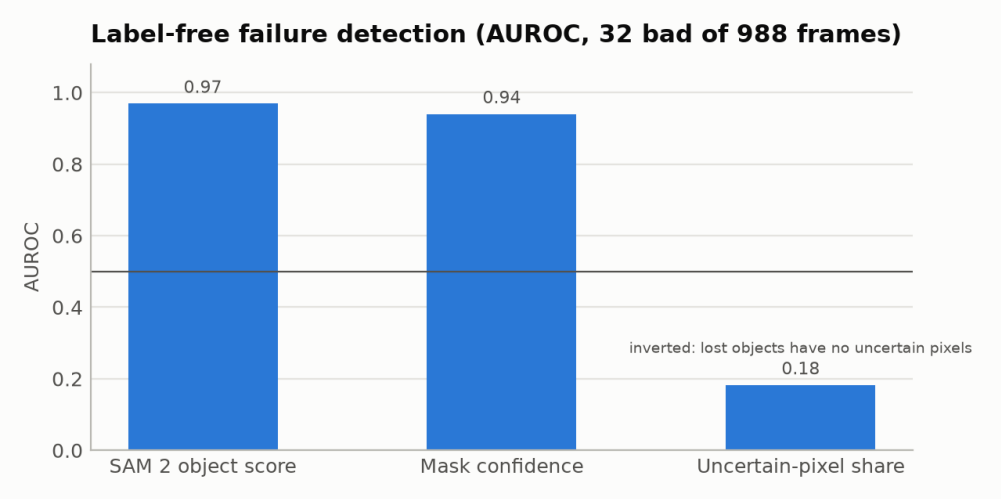


Figure 5 – How well each label-free signal separates failed frames from good ones (0.5 = chance).

Object	Temporal IoU	Aligned IoU	Flow-warped IoU	Empty frames	Isolated drop-outs	Min object score	Confi-dence
car-shadow #1	0.95	0.97	0.98	0	0	7.66	0.990
synthetic/occlusion #1	0.72	0.85	0.93	12	0	-5.21	0.993
synthetic/occlusion #2	0.86	0.98	0.98	0	0	9.07	0.992
synthetic/exit_reentry #1	0.51	0.93	0.86	27	0	-5.43	0.994
synthetic/fast_motion #1	0.20	0.85	0.29	0	0	2.67	0.970
synthetic/distractor #1	0.73	0.93	0.92	0	0	2.48	0.990
sav_000001 #1	0.06	0.65	0.22	5	1	-2.99	0.977
sav_000001 #2	0.42	0.61	0.51	0	0	7.55	0.979
sav_000001 #3	0.56	0.75	0.75	9	4	-4.59	0.972
sav_000001 #4	0.08	0.50	0.26	0	0	5.65	0.975
sav_000001 #5	0.10	0.69	0.19	0	0	5.23	0.979
bedroom #1 (no GT)	0.49	–	0.70	0	0	8.21	0.975
bedroom #2 (no GT)	0.38	–	0.52	0	0	3.27	0.962

Raw temporal IoU mixes real motion with mask instability. Aligned IoU removes translation by matching centroids, so what remains is change of shape. Flow-warped IoU removes all motion when optical flow works, but it breaks down at large displacements (see the fast-motion and 6 fps rows). A negative object score marks frames where SAM 2 itself says the object is not visible. The bedroom row has no aligned IoU because its masks were not stored.

Do the rule-based alerts fire on the right runs?

Run / object	J&F	Aligned IoU	Alerts (default thresholds)
car-shadow (mask, s1) #1	0.987	0.97	none
car-shadow (box, s1) #1	0.987	0.97	none
car-shadow (click, s1) #1	0.987	0.97	none
car-shadow (click3, s1) #1	0.987	0.97	none

Run / object	J&F	Aligned IoU	Alerts (default thresholds)
car-shadow (mask, s2) #1	0.987	0.93	none
car-shadow (mask, s4) #1	0.988	0.87	none
synthetic/occlusion (mask, s1) #1	0.990	0.85	area_jump
synthetic/occlusion (mask, s1) #2	0.986	0.98	none
synthetic/exit_reentry (mask, s1) #1	0.672	0.93	object_lost
synthetic/fast_motion (mask, s1) #1	0.640	0.85	none
synthetic/distractor (mask, s1) #1	0.976	0.93	none
synthetic/distractor (click, s1) #1	0.975	0.93	none
sav_000001 (mask, s4) #1	0.872	0.65	flicker, frame_dropout, area_jump
sav_000001 (mask, s4) #2	0.971	0.61	flicker
sav_000001 (mask, s4) #3	0.912	0.75	frame_dropout, area_jump
sav_000001 (mask, s4) #4	0.997	0.50	flicker, area_jump
sav_000001 (mask, s4) #5	0.967	0.69	flicker, area_jump
synthetic/fast_motion (mask, s2) #1	0.634	0.76	area_jump
synthetic/fast_motion (mask, s4) #1	0.693	0.71	area_jump
synthetic/occlusion (mask, s2) #1	0.990	0.78	area_jump
synthetic/occlusion (mask, s2) #2	0.988	0.98	none
synthetic/occlusion (mask, s4) #1	0.993	0.70	flicker, area_jump
synthetic/occlusion (mask, s4) #2	0.987	0.98	none
bedroom (no GT) #1	n/a	n/a	none
bedroom (no GT) #2	n/a	n/a	flicker, area_jump

The rule-based monitor caught the clear failure (exit and re-entry → `object_lost`). It cannot see the fast-motion failure at stride 1: there the mask is consistently too large, and stability metrics only detect *changes*, not a constant bias. It also raises alerts on runs that are fine: `area_jump` when the visible area legitimately changes under partial occlusion, and `flicker` / `frame_dropout` on the 6 fps SA-V clip, where articulated objects really do change shape between processed frames and single-frame absences can be correct. Read the alerts as triage signals for a human reviewer, calibrated per frame rate and footage type. For per-frame decisions such as automatic re-prompting, the object score is the precise signal. One more lesson: flow-warped IoU collapsed (0.0–0.3) at large displacements where tracking was fine, because dense optical flow fails there. That is why the flicker rule now uses centroid-aligned IoU first.

Cost model on this machine

Quantity	Value
Image encoder (Hiera-T, 1024^2) per frame	2,284 ms
Memory attention + decoder per object per frame	3,557 ms
Fixed per-frame overhead (fit intercept)	≈ 0 (fit: -322 ms, within noise)
Measured, 1 object(s), encoder cached	3,328 ms/frame
Measured, 2 object(s), encoder cached	6,654 ms/frame
Measured, 4 object(s), encoder cached	13,953 ms/frame
Projected throughput, 1 object(s)	0.18 fps

Quantity	Value
Projected throughput, 2 object(s)	0.11 fps
Projected throughput, 4 object(s)	0.06 fps
Projected throughput, 8 object(s)	0.03 fps
Projected throughput, 16 object(s)	0.02 fps

The model has two cost blocks. The image encoder runs once per frame (≈ 2.3 s here). Memory attention + mask decoder run once per object per frame (≈ 3.6 s here; the fitted fixed overhead is ≈ 0). With 4 objects that is ≈ 14 s per frame on 2 vCPU, so SAM 2.1 runs every object separately and cost grows linearly. That is the bottleneck SAM 3.1's Object Multiplex removes (up to 16 objects in one pass). On a GPU both blocks shrink by one to two orders of magnitude. The feature cache removes the encoder block whenever the same video is re-run with new prompts or strides, which is the normal interactive-roto loop (click, inspect, add a correction, re-propagate).

3.7 Threats to validity

- **Small real-data sample.** One DAVIS sequence, and one SA-V video that belongs to SAM 2's training split. Headline numbers need the full DAVIS 2017 val / MOSEv2. The harness supports them; only the download was blocked.
- **Oracle prompts.** Clicks are derived from ground truth (deepest interior point). Real users click less precisely.
- **Synthetic domain gap.** The stress clips isolate failure modes; they do not predict real-world rates.
- **Weights.** A public fp16 mirror of SAM 2.1 tiny, upcast to fp32. Expect small differences from Meta's fp32 file.
- **Timing noise.** A shared 2-vCPU sandbox (±20 %). Accuracy numbers are deterministic and unaffected.
- **Monitor thresholds** were set on the same small benchmark they are evaluated on. Recalibrate them on production traffic before trusting alert rates.

4. Architecture: what changed in the base platform

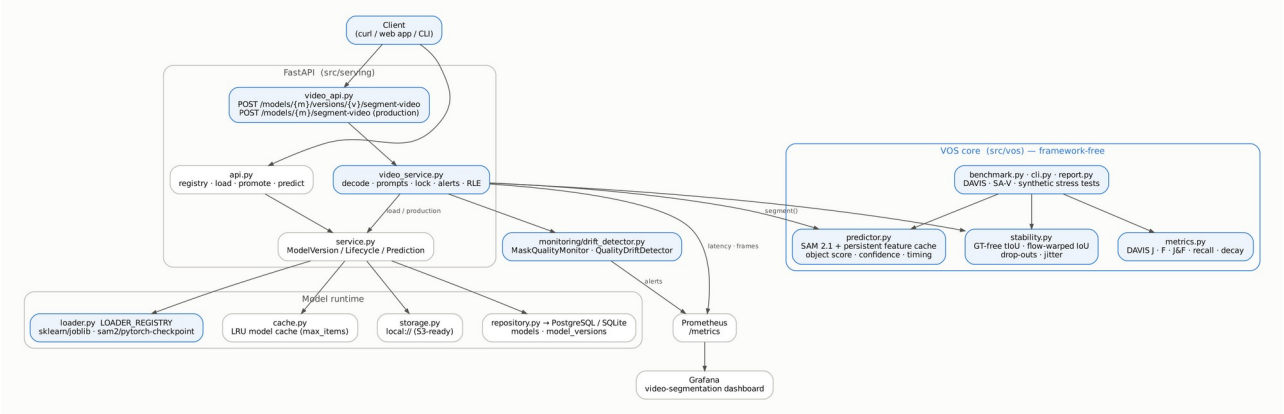


Figure 6 – Blue = added or extended in this phase; grey = reused from the ML Model Serving Platform.

Concern	Before (MSP)	After (this phase)
Model runtimes	scikit-learn/joblib, hard-coded in the service	Loader registry: scikit-learn/joblib + SAM 2/pytorch-checkpoint (variant auto-detected)
Inference API	`/predict` with a feature vector	+ multipart `/segment-video` (versioned and production), COCO-RLE masks, stability report, alerts
Model cache	Unbounded dict	Thread-safe LRU with `max_items`
Database	PostgreSQL only	PostgreSQL or SQLite (`DATABASE_URL`); partial index on both

Concern	Before (MSP)	After (this phase)
Observability	Tabular prediction metrics	+ video latency/frames, temporal IoU, empty-frame ratio, quality alerts; new Grafana dashboard
Monitoring	`drift_detector.py` was an empty placeholder	MaskQualityMonitor (per request) + QualityDriftDetector (across requests)
Tests / CI	63 tests, only pass against a pre-initialised DB	122 tests; isolated DB; CI lint + PostgreSQL + SAM 2 smoke
Container	Single stage, root, no .dockerignore	Multi-stage, non-root, CPU torch, pinned SAM 2 commit

File inventory

File	Status	Lines (now / base)
`src/models/seed.py`	unchanged	61 / 61
`src/models/seed_sam2.py`	new	51
`src/models/train.py`	unchanged	167 / 167
`src/monitoring/drift_detector.py`	modified	106
`src/serving/api.py`	modified	166 / 159
`src/serving/app.py`	modified	61 / 44
`src/serving/cache.py`	modified	78 / 51
`src/serving/config.py`	modified	50 / 23
`src/serving/database.py`	modified	25 / 21
`src/serving/db_models.py`	modified	97 / 85
`src/serving/dependency.py`	unchanged	8 / 8
`src/serving/loader.py`	modified	110 / 52
`src/serving/observability.py`	modified	193 / 105
`src/serving/repository.py`	unchanged	95 / 95
`src/serving/schemas.py`	modified	147 / 54
`src/serving/service.py`	modified	219 / 210
`src/serving/storage.py`	unchanged	79 / 79
`src/serving/video_api.py`	new	124
`src/serving/video_service.py`	new	197
`src/vos/__init__.py`	new	19
`src/vos/benchmark.py`	new	409
`src/vos/cli.py`	new	544
`src/vos/data.py`	new	190
`src/vos/metrics.py`	new	268
`src/vos/predictor.py`	new	341
`src/vos/prompts.py`	new	68
`src/vos/report.py`	new	537
`src/vos/rle.py`	new	113

File	Status	Lines (now / base)
src/vos/stability.py	new	222
src/vos/synthetic.py	new	211
src/vos/types.py	new	110
src/vos/visualize.py	new	103
tests/conftest.py	new	32
tests/test_health.py	unchanged	22 / 22
tests/test_infrastructure.py	unchanged	57 / 57
tests/test_lifecycle.py	unchanged	244 / 244
tests/test_loader.py	unchanged	172 / 172
tests/test_observability.py	unchanged	231 / 231
tests/test_prediction.py	modified	228 / 228
tests/test_registry.py	unchanged	75 / 75
tests/test_sam2_integration.py	new	136
tests/test_storage.py	unchanged	67 / 67
tests/test_video_api.py	new	225
tests/test_vos_metrics.py	new	99
tests/test_vos_rle.py	new	44
tests/test_vos_stability_prompts.py	new	126
tests/test_vos_synthetic.py	new	49

5. Code review

5.1 Method and severity scale

Every file was read line by line. Findings are rated as follows:

Severity	Meaning
High	Security exposure or a defect that breaks a documented workflow (e.g. CI on a fresh database).
Medium	Wrong behaviour under realistic load or inputs; design that blocks the next feature.
Low	Maintainability, consistency, small inefficiencies.
Info	Strengths and design notes worth keeping.

Status: **Fixed** means changed and tested in this phase; **Open** means a recommendation with a concrete next step; **By design** means an accepted trade-off.

5.2 Base platform (reviewed, then extended)

Repository packaging (zip, .env, .venv, .idea, bk/)

Purpose	Technique
How the project is shared and built.	Git-ignored local artefacts; Docker build context.

Severity	Finding	Resolution / status
High	The shared archive contains `.env` with real database and Grafana passwords. It is git-ignored, but the zip bypasses that.	**Fixed:** excluded from the new project; only `env.example` ships. Rotate those passwords.
Medium	The archive ships `.venv` (484 MB, 33 k files), `.idea`, `.ruff_cache`, `__pycache__` and the `bk/` scratch folder. There is no `.dockerignore`, so `COPY . .` would copy the venv into the image.	**Fixed:** `.dockerignore` added; clean export.

src/serving/config.py + database.py

Purpose	Technique
Settings from environment; SQLAlchemy engine and session factory.	pydantic-settings `BaseSettings`; dependency-injected `get_db()` session.

Severity	Finding	Resolution / status
Medium	The DB URL is hard-wired to PostgreSQL and all `POSTGRES_*` fields are required, so nothing (not even unit tests) runs without Docker.	**Fixed:** `DATABASE_URL` override (SQLite for local / tests), `check_same_thread=False` for SQLite, `pool_pre_ping=True`.
Low	Inconsistent field casing (`ARTIFACT_STORAGE_BACKEND` vs `postgres_user`); `env.example` used `KEY = 'value'` with spaces and quotes.	**Fixed:** `env.example` normalised; field names kept for backward compatibility.
Info	New settings: `model_cache_max_items`, `vos_device`, `vos_max_frames`, `vos_max_upload_mb`, `vos_num_threads`.	**Added**.

src/serving/db_models.py + repository.py

Purpose	Technique
Registry tables (`models`, `model_versions`, scaffolded `model_deployments`) and the only place that issues SQL.	Partial unique index = "at most one production version per model", enforced by the database. Promotion and archiving run in one transaction.

Severity	Finding	Resolution / status
Low	The partial index only had `postgresql_where`, so the guarantee disappeared on any other engine.	**Fixed:** matching `sqlite_where`.
Low	`status` is free text; `DBModelDeployment` (canary) is unused scaffolding; tables come from `create_all()` with no migrations.	**Open:** DB Enum/CHECK, Alembic, implement or remove the canary table.
Info	Strength: the transactional promote/rollback with a DB-level invariant is production-grade and was reused unchanged for SAM 2 versions.	**By design**

src/serving/loader.py

Purpose	Technique
Turn artifact bytes into an in-memory model.	`Protocol` interface + **loader registry** keyed by `(framework, model_format)`.

Severity	Finding	Resolution / status
Medium	The service layer hard-coded `if framework != 'scikit-learn'`. That contradicts the README's own claim that the platform is algorithm-agnostic; adding PyTorch meant editing business logic.	**Fixed:** `LOADER_REGISTRY` + `get_loader()`; a new runtime is one registry entry. `Sam2CheckpointLoader` added.
High	`joblib.load` (pickle) executes arbitrary code. Anyone who can register an `artifact_uri` and write to storage can run code in the API process.	**Open:** store an artifact SHA-256 in the registry and verify it at load time; add auth on registry endpoints; prefer skops/ONNX. The SAM 2 path already uses `torch.load(weights_only=True)`.
Info	SAM 2 variant (tiny / small / base+ / large) is inferred from tensor shapes, so the registry needs no schema change.	**Added**

src/serving/service.py

Purpose	Technique
Business rules: registry, loading, lifecycle (promote / production), prediction.	Three-layer architecture (API → Service → Repository).

Severity	Finding	Resolution / status
Medium	Calling `/predict` on a model without `.predict` (a video model) caused an `AttributeError`, which surfaced as a 500.	**Fixed:** `TaskMismatchError` → 400 with a clear message.
Low	Latency was measured with `time.time()` (wall clock, can jump under NTP adjustments).	**Fixed:** `time.perf_counter()`.

Severity	Finding	Resolution / status
Low	The service raises FastAPI `HTTPException`, so HTTP concerns leak into the business layer. The rest of the code uses domain exceptions.	**Open:** domain exceptions + `app.exception_handler`.
Low	Promotion loads the full model only to check that it loads. For SAM 2 that means seconds and hundreds of MB.	**By design** (safety first). Option: cache the validated model after a successful promotion.

src/serving/api.py

Purpose	Technique
REST routes for the registry, lifecycle and tabular prediction.	FastAPI routers, Pydantic I/O models, explicit exception → status-code mapping, labelled error metrics.

Severity	Finding	Resolution / status
Low	`POST /models/{name}/predict` accepts a raw `dict` instead of `PredictionRequest`. Validation differs from the versioned route and errors are not counted in metrics.	**Open:** reuse the schema and the same error mapping.
Low	POST returns 200 instead of 201; no pagination or auth (already listed in the original README).	**Open**

src/serving/cache.py

Purpose	Technique
Keep hot models in RAM.	`ModelCache` protocol; process-local singleton.

Severity	Finding	Resolution / status
Medium	The cache was an unbounded `dict`. That is fine for kilobyte scikit-learn models, but a few SAM 2 checkpoints (80 MB – 900 MB each) would exhaust memory.	**Fixed:** LRU with `max_items` (default 4), `RLock` for thread-pool safety, eviction counter. `max_items=None` keeps the old behaviour.
Low	The cache is per process: N uvicorn workers hold N copies of each model.	**By design:** torch modules cannot live in Redis. Use one worker per GPU, or a model server (Triton / TorchServe) when scaling out.

src/serving/storage.py + dependency.py

Purpose	Technique
Artifact storage abstraction (`local://`, S3-ready).	Path-traversal protection via `os.path.commonpath`; DI factory.

Severity	Finding	Resolution / status
Info	Strength: the traversal check is correct, and the tests cover it.	**By design**
Low	`get()` reads the whole artifact into memory. That is acceptable for an 80 MB checkpoint but wasteful for 900 MB; a new `LocalArtifactStorage` object is built on every call.	**Open:** streaming / `local_path()` API; cache the storage instance.

src/serving/observability.py

Purpose	Technique
Prometheus metrics, structured JSON logs, request IDs via `ContextVar`.	Counters and histograms labelled by model and version; no PII in logs.

Severity	Finding	Resolution / status
Medium	Default histogram buckets (5 ms – 10 s) suit tabular inference only; video requests take seconds to minutes and would all land in +Inf.	**Fixed:** explicit buckets for `video_segmentation_latency_seconds` (0.5 s – 600 s) and `video_frame_latency_seconds`.
Info	New metrics: video requests / errors / latency / frames, `mask_temporal_iou`, `mask_empty_frame_ratio`, `mask_quality_alerts_total{reason}`.	**Added**

deployment/Dockerfile + docker-compose.yml

Purpose	Technique
Container image and local stack (api · postgres · prometheus · grafana).	Health checks; Grafana provisioning as code.

Severity	Finding	Resolution / status
Medium	The image was single-stage, ran as root, and shipped `build-essential` at runtime.	**Fixed:** multi-stage build, non-root user, CPU PyTorch wheels, pinned SAM 2 commit. Not built in this environment (no Docker daemon).
Low	The README's `--shm-size=2gb` advice was not in compose; prometheus and grafana use `latest` tags.	**Fixed:** `shm_size: 2gb`; weights volume. Open: pin monitoring images.

tests/ + .github/workflows/ci-cd.yaml

Purpose	Technique
8 pytest suites (63 tests) and lint/test CI.	Integration tests through `TestClient`; mocking for cache hits.

Severity	Finding	Resolution / status
High	Tables are created in the FastAPI lifespan, but the tests use `TestClient(app)` without a context manager, so the lifespan never runs. The suite only passed because the running API container had already created the tables. On the fresh PostgreSQL in CI every DB test fails.	**Fixed:** `tests/conftest.py` creates the tables once per session and defaults to an isolated temporary SQLite database.
Low	One test patched `src.serving.service.SklearnJoblibLoader`, which ties it to an import location.	**Fixed:** it now patches the loader module.
Info	CI now runs lint, the fast suite (PostgreSQL service) and a push-only SAM 2 CPU smoke job.	**Added**

5.3 New video-segmentation code

src/vos/predictor.py

Purpose	Technique
SAM 2.1 video segmenter behind a small `VideoSegmenter` protocol.	Streaming memory propagation; prompts as mask, box or clicks; late-appearing objects tracked in a separate pass (same policy as Meta's `vos_inference.py`). A **persistent backbone-feature cache** reuses Hiera features across runs on the same video. SAM 2's object-presence logit and per-pixel probabilities are exposed as GT-free confidence signals.

Severity	Finding	Resolution / status
Info	The feature cache returns bit-identical masks (verified in a test). Position encodings (≈80 MB of the ≈100 MB per frame) are deduplicated, so a cached frame costs ≈20 MB.	By design
Medium	`init_state` loads every frame as a float32 1024² tensor (≈12.6 MB/frame). Measured peak RSS: 5.6 GB for 121 frames × 5 objects (frames + memory bank + feature cache).	**Mitigated:** API limit `vos_max_frames=300`, `stride`. Open: windowed / streaming frame loading for long videos.
Medium	The predictor keeps per-call state on the instance, so one model instance must not run two videos at once.	**Mitigated:** per-model lock in `video_service.py`. Open: job queue + worker pool for throughput.
Low	The optional CUDA extension `sam2._C` is not built on CPU, so the hole-filling post-processing is skipped (Meta notes the effect is small).	By design on CPU; build the extension in the GPU image.
Low	The weights come from a public mirror stored in fp16 and upcast to fp32. Scores may differ slightly from Meta's fp32 release.	**Open:** `fetch-weights` tries Meta's URL first; the SHA-256 is recorded in `runs.json`.

src/vos/metrics.py

Purpose	Technique
DAVIS region similarity J, contour accuracy F, J&F, recall and decay.	Port of `davis2017-evaluation`: one-pixel boundary maps (`seg2bmap`) dilated by a disk of radius ⌈0.008 × diagonal⌉; empty-vs-empty scores 1. Supports sparse GT (SA-V, 6 fps) and stride sub-sampling.

Severity	Finding	Resolution / status
Medium	Fixed an upstream bug: the official `db_statistics` casts decay-bin indices to `uint8`, so decay is wrong for videos longer than 255 frames (tested).	**Fixed**
Low	Not yet cross-checked against the official toolkit on full DAVIS 2017 val (the dataset host is not reachable from this environment).	**Open:** run `davis2017-evaluation` on the same predictions.

src/vos/stability.py

Purpose	Technique
Ground-truth-free temporal metrics per tracked object.	Temporal IoU; **centroid-aligned IoU** (translation removed, no flow needed); **flow-compensated IoU** (Farnebäck backward flow; warp mask t into t+1 and compare); area volatility; isolated drop-outs; lost / recovered events; centroid jitter; confidence and object score.

Severity	Finding	Resolution / status
Medium	Flow-warped IoU collapsed to 0.0–0.3 at large displacements (fast motion, 6 fps SA-V) while tracking was fine, because dense optical flow fails there. Raw temporal IoU has the same problem for any fast object.	**Fixed:** added centroid-aligned IoU; the monitor uses it first.
Low	Stability metrics detect *change*, not a constant bias. A mask that is consistently too large (motion blur) looks perfectly stable.	**By design:** pair with the object score and periodic GT spot checks.
Low	Flow is computed at half resolution, ≈40 ms per frame pair at 480p on CPU, so it is opt-in in the API (`flow_stability`).	**By design**

src/vos/synthetic.py

Purpose	Technique
Stress-test videos with exact (modal) ground truth.	Procedural textured background and clutter; striped sprites with shading; foreground occluder; identical distractor drawn in front; exit / re-entry; 50 px/frame zig-zag with 7-sample motion blur. JPEG-encoded like real footage.

Severity	Finding	Resolution / status
Low	Rendering builds a full-frame mesh grid per sprite (≈0.15 s/frame).	**Open:** render inside the sprite's bounding box.
Info	There is a domain gap to real video. These clips isolate failure modes; they do not replace a real benchmark.	**By design**

src/vos/prompts.py

Purpose	Technique
Derive mask, box, 1-click and 3-click prompts from GT.	First click at the maximum of the distance transform (always inside, even for rings); extra clicks by farthest-point sampling over deep interior pixels.

Severity	Finding	Resolution / status
Low	Oracle prompts are cleaner than real users' clicks.	**Open:** add click jitter / noisy boxes in Phase 2.

src/vos/data.py + rle.py

Purpose	Technique
DAVIS (palette PNG, void = 255), SA-V train format (compressed COCO RLE) and unlabeled-video loaders; frame materialisation for SAM 2.	Pure-NumPy port of pycocotools' compressed RLE (LEB128-like 5-bit groups, delta-to-two-back), verified against pycocotools. Frames are hard-linked, not copied.

Severity	Finding	Resolution / status
Info	No C build dependency for masks; the API returns uncompressed RLE (column-major), which any COCO tool can read.	**By design**
Low	For long clips, uncompressed RLE in JSON is large.	**Open:** offer compressed RLE strings or a ZIP of PNGs.

src/vos/benchmark.py + cli.py + report.py

Purpose	Technique
Config-driven experiments, CSV/JSON results, plots, demo media.	Run matrix (sequence × prompt × stride); event metrics (hidden-frame J, re-detection delay, distractor leakage, identity switch); AUROC / Spearman of GT-free signals against J; `--append` to add experiments; `profile` cost model.

Severity	Finding	Resolution / status
Low	Runtime numbers come from a shared 2-vCPU sandbox (±20 %). Accuracy numbers are deterministic.	**By design:** `profile` measures the cost model on a quiet machine.
Info	The run environment (torch version, threads, checkpoint SHA-256 and dtype) is stored with the results.	**Added**

src/serving/video_api.py + video_service.py

Purpose	Technique
`POST /models/{m}/versions/{v}/segment-video` and production routing.	Multipart upload streamed to disk; size and frame guards (413); prompt validation (422 for unprocessed frames or points outside the frame); registry → LRU cache → SAM 2; per-model lock; stability report + alerts + Prometheus; COCO RLE masks.

Severity	Finding	Resolution / status
Medium	The endpoint is synchronous. A 300-frame clip on CPU takes minutes and holds an HTTP worker.	**Open:** job queue (Celery / RQ / Arq) with the same service function as the worker body; GPU node.
Low	The upload-size limit is checked after the file is written.	**Open:** enforce at the proxy (`client_max_body_size`) or count bytes while streaming.

src/monitoring/drift_detector.py

Purpose	Technique
Per-request quality rules and cross-request drift detection without labels.	Threshold rules map to client-visible failures (flicker, object lost, frame drop-out, area jump, low confidence). One-sided mean-shift z-test of a quality signal: reference window vs recent window.

Severity	Finding	Resolution / status
Medium	On the benchmark the default rules caught the clear failure (exit and re-entry) but also alerted on good runs: `area_jump` under legitimate partial occlusion, and `flicker` / `frame_dropout` on the 6 fps SA-V clip (§3.6).	**Open:** calibrate per frame rate and footage type; gate `area_jump` on the object score; treat alerts as triage.
Low	Thresholds come from a small benchmark, and the z-test assumes independent requests.	**Open:** recalibrate on production traffic; consider CUSUM / Page-Hinkley.

5.4 Key code, explained

Feature cache hook (predictor.py)

SAM 2 keeps the backbone features of one frame only. The hook below serves features from a persistent LRU keyed by (video, original frame index), so mask, box and click runs, and different strides, share one encoder pass.

On a hit it rebuilds the image view from the current inference state, so the previous run's frame tensor is not kept alive.

```
        def cached_get_image_feature(inference_state, frame_idx, batch_size):
            ids = segmenter._frame_ids
            key = (segmenter._video_key, ids[frame_idx] if ids is not None else frame_idx)
            backbone_out = cache.get(key) if segmenter._video_key else None
            if backbone_out is not None:
                # Re-use the features but take the image view from the *current* state:
                # caching the image would pin the previous run's whole frame tensor in RAM.
                device = inference_state["device"]
                image = inference_state["images"][frame_idx].to(device).float().unsqueeze(0)
                inference_state["cached_features"] = {frame_idx: (image, backbone_out)}
            out = original(inference_state, frame_idx, batch_size)
            if backbone_out is None and segmenter._video_key:
                cache.put(key, inference_state["cached_features"][frame_idx][1])
            return out
```

Object-presence score (predictor.py)

Besides the mask, the SAM 2 decoder predicts whether the object is visible at all. Reading it from the inference state gives a free occlusion detector (see §3.6).

```
    def _object_score(state, obj_id: int, frame_idx: int) -> float | None:
        """SAM 2's object-presence logit for (object, frame); > 0 means 'object visible'."""
        try:
            obj_idx = state["obj_id_to_idx"][obj_id]
            out = state["output_dict_per_obj"][obj_idx]
            d = out["cond_frame_outputs"].get(frame_idx) or
out["non_cond_frame_outputs"].get(frame_idx)
            if d is None or d.get("object_score_logits") is None:
                return None
```

Contour accuracy F (metrics.py)

```
def f_measure(gt: np.ndarray, pred: np.ndarray, void: np.ndarray | None = None, bound_th: float =
0.008) -> float:
    """Boundary F-measure with a distance tolerance (port of DAVIS ``f_measure``)."""
    gt = gt.astype(bool)
    pred = pred.astype(bool)
    if void is not None:
        keep = ~void.astype(bool)
        gt = gt & keep
        pred = pred & keep

    bound_pix = boundary_tolerance_px(gt.shape, bound_th)
    fg_boundary = seg2bmap(pred)
    gt_boundary = seg2bmap(gt)
    kernel = _disk(bound_pix)
    fg_dil = cv2.dilate(fg_boundary.astype(np.uint8), kernel).astype(bool)
    gt_dil = cv2.dilate(gt_boundary.astype(np.uint8), kernel).astype(bool)

    gt_match = gt_boundary & fg_dil
    fg_match = fg_boundary & gt_dil
    n_fg = int(np.count_nonzero(fg_boundary))
    n_gt = int(np.count_nonzero(gt_boundary))

    if n_fg == 0 and n_gt > 0:
        precision, recall = 1.0, 0.0
    elif n_fg > 0 and n_gt == 0:
        precision, recall = 0.0, 1.0
    elif n_fg == 0 and n_gt == 0:
```

```
        precision, recall = 1.0, 1.0
    else:
        precision = np.count_nonzero(fg_match) / n_fg
        recall = np.count_nonzero(gt_match) / n_gt

    if precision + recall == 0:
        return 0.0
    return float(2 * precision * recall / (precision + recall))
```

Decay with the uint8 fix (metrics.py)

```
def statistics(per_frame: np.ndarray) -> Stat:
    """Mean, recall (>0.5) and decay of a per-frame score vector (NaNs ignored)."""
    v = np.asarray(per_frame, dtype=float)
    if v.size == 0 or np.all(np.isnan(v)):
        return Stat(float("nan"), float("nan"), float("nan"))
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", category=RuntimeWarning)
        mean = float(np.nanmean(v))
        recall = float(np.nanmean(v[~np.isnan(v)] > 0.5))
        n_bins = 4
        # FIX vs. official toolkit: int64 instead of uint8 (overflow for > 255 frames)
        ids = (np.round(np.linspace(1, len(v), n_bins + 1) + 1e-10) - 1).astype(np.int64)
        bins = [v[ids[i] : ids[i + 1] + 1] for i in range(n_bins)]
        decay = float(np.nanmean(bins[0]) - np.nanmean(bins[3]))
    return Stat(mean, recall, decay)
```

Motion-compensated stability (stability.py)

Two ways to take motion out of temporal IoU: shift the previous mask so the centroids coincide (cheap and robust), or warp it with dense backward optical flow (exact when the flow is right, useless when the displacement is too large).

```
def aligned_iou(a: np.ndarray, b: np.ndarray) -> float:
    """IoU after translating ``a`` so that its centroid coincides with ``b``'s (integer
pixels)."""
    ya, xa = np.nonzero(a)
    yb, xb = np.nonzero(b)
    if xa.size == 0 or xb.size == 0:
        return float("nan")
    dy = int(round(yb.mean() - ya.mean()))
    dx = int(round(xb.mean() - xa.mean()))
    h, w = a.shape
    ys, xs = ya + dy, xa + dx
    keep = (ys >= 0) & (ys < h) & (xs >= 0) & (xs < w)
    shifted = np.zeros_like(a, dtype=bool)
    shifted[ys[keep], xs[keep]] = True
    return _iou(shifted, b)

def warp_mask(mask_t: np.ndarray, flow_back: np.ndarray) -> np.ndarray:
    """Warp mask_t into frame t+1 using backward flow (t+1 → t)."""
    h, w = mask_t.shape
    gx, gy = np.meshgrid(np.arange(w, dtype=np.float32), np.arange(h, dtype=np.float32))
    map_x = gx + flow_back[..., 0].astype(np.float32)
    map_y = gy + flow_back[..., 1].astype(np.float32)
    warped = cv2.remap(mask_t.astype(np.float32), map_x, map_y, cv2.INTER_LINEAR, borderValue=0)
    return warped > 0.5
```

Loader registry (loader.py)

```
LOADER_REGISTRY: dict[tuple[str, str], Any] = {
    ("scikit-learn", "joblib"): SklearnJoblibLoader,
    ("sam2", "pytorch-checkpoint"): _sam2_factory,
```

```
}

# what each framework is able to do (used to reject e.g. /predict on a video model)
TASK_BY_FRAMEWORK = {"scikit-learn": "tabular-prediction", "sam2": "video-segmentation"}


def get_loader(framework: str, model_format: str) -> ModelLoader:
    factory = LOADER_REGISTRY.get((framework, model_format))
    if factory is None:
        supported = ", ".join(f"{f}/{m}" for f, m in LOADER_REGISTRY)
        raise UnsupportedModelFormat(f"Unsupported format {framework}/{model_format} (supported:
{supported})")
    return factory()
```

Video endpoint guard-rails (video_service.py)

```
def _to_prompts(req: VideoSegmentationRequest, meta: dict) -> dict[int, Prompt]:
    pos_of = {orig: k for k, orig in enumerate(meta["original_indices"])}
    h, w = meta["height"], meta["width"]
    prompts: dict[int, Prompt] = {}
    for o in req.objects:
        if o.frame_idx not in pos_of:
            raise InvalidPromptError(
                f"object {o.obj_id}: frame_idx {o.frame_idx} is not processed "
                f"(stride={req.stride}, processed frames are multiples of the stride)"
            )
        coords = (o.points or []) + ([o.box[:2], o.box[2:]] if o.box else [])
        for x, y in coords:
            if not (0 <= x <= w and 0 <= y <= h):
                raise InvalidPromptError(f"object {o.obj_id}: point ({x}, {y}) is outside the
{w}x{h} frame")
```

Quality rules (drift_detector.py)

```
    def evaluate(self, report: dict) -> list[str]:
        """Return the list of alert reasons for one object's stability report."""
        t = self.t
        alerts: list[str] = []

        def val(key):
            v = report.get(key)
            return None if v is None or (isinstance(v, float) and math.isnan(v)) else v

        # Flicker = the mask's *shape* changes between frames. Raw temporal IoU also drops for
fast
        # but perfectly tracked objects (false alarms on sports / kids footage), so use a
        # motion-compensated estimate. Centroid-aligned IoU comes first: in the Phase-1
benchmark,
        # flow-warped IoU collapsed (0.0–0.3) at large displacements where tracking was fine,
        # because dense optical flow itself fails there.
        for key, threshold in (
            ("aligned_iou_mean", t.min_aligned_iou),
            ("warped_iou_mean", t.min_warped_iou),
            ("tiou_mean", t.min_tiou),
        ):
            if val(key) is not None:
                if val(key) < threshold:
                    alerts.append("flicker")
                break
        if val("empty_rate") is not None and val("empty_rate") > t.max_empty_rate:
            alerts.append("object_lost")
```

```
        if (val("isolated_dropouts") or 0) > t.max_isolated_dropouts:
            alerts.append("frame_dropout")
        if val("area_jump_rate") is not None and val("area_jump_rate") > t.max_area_jump_rate:
            alerts.append("area_jump")
        if val("confidence_mean") is not None and val("confidence_mean") < t.min_confidence:
            alerts.append("low_confidence")
        return alerts
```

6. Techniques explained (learning notes)

SAM 2: promptable segmentation with streaming memory

Each frame is encoded once by a hierarchical vision transformer (Hiera). Memory attention then cross-attends the current frame's features to a memory bank. The bank holds the prompted (conditioning) frames, the most recent frames (6 by default) and compact object pointers. The mask decoder outputs a mask, an IoU estimate and an object-presence score.

The memory is why a mask persists: the model does not re-detect from scratch, it re-identifies what it has already seen. The fixed recent-frame window is also why long absences are hard; the exit-and-re-entry case shows it.

Semi-supervised VOS protocol and metrics

Prompt once, propagate, then score every annotated frame except the first and last. J is IoU; F is a boundary F-measure with a distance tolerance. J&F averages the two, recall is the share of frames above 0.5, and decay compares the first and last quarter of the video. Decay is the most direct answer to “does the mask persist?”.

GT-free stability and flow compensation

Without labels you can still measure consistency. Temporal IoU between consecutive masks penalises flicker but also real motion. Warping the previous mask with dense optical flow (Farnebäck, backward flow + `cv2.remap`) before comparing removes the motion term, so the remaining disagreement is segmentation instability. Drop-outs (a single empty frame between visible ones) are the “frame drop” a roto client complains about.

Feature caching

The image encoder is about 40 % of the per-frame cost for one object on CPU (2.3 of 5.6 s) and does not depend on the prompt. Caching its output per (video, frame) turns every extra prompt experiment, and every interactive correction, into a memory-attention-only pass. Results are bit-identical; the cache deduplicates the input-independent position encodings to cut memory 5×.

Segmentation propagation vs. tracking-by-detection

Tracking-by-detection (SORT, ByteTrack, OC-SORT) runs a detector on every frame and associates boxes over time with a Kalman motion model and IoU or appearance matching. It finds new objects automatically and recovers after exits, but outputs boxes and suffers ID switches. SAM 2 propagation needs a prompt, outputs pixel masks and keeps identity through appearance memory, but it does not discover new objects and can stay lost after long absences.

The two are complementary. The hybrid in Phase 2 lets a detector re-prompt SAM 2 whenever the object score indicates loss. Multi-object variants of the same idea are SAM2MOT, and SAM 3/3.1 with text prompts.

Serving patterns used

Model registry with a DB-enforced single production version; a loader registry for pluggable runtimes; a bounded LRU model cache; per-model locks around stateful predictors; explicit Prometheus buckets for long-running requests; COCO RLE for compact, tool-agnostic mask transport; rule-based quality alerts plus a rolling mean-shift detector for label-free drift.

7. Recommendations and roadmap

7.1 Prioritised fixes

1. Verify an artifact hash at load time and add auth on registry endpoints (pickle RCE exposure, inherited from the base platform).
2. Move video segmentation to a job queue with GPU workers; keep the synchronous endpoint for short clips.

3. Add the detector re-prompt hybrid, triggered by the object score, to fix the exit and re-entry failure.
4. Stream frames instead of loading the whole clip (RAM grows by 12.6 MB per frame today).
5. Run the full DAVIS 2017 val and MOSEv2 and cross-check metrics with the official toolkit.
6. Replace binary masks with soft alpha for motion-blurred objects (matting stage).

7.2 Next phases

1. Phase 2: YOLO-seg + ByteTrack vs. SAM 2 vs. a hybrid on the same clips; HOTA and IDF1 next to J&F.
2. Phase 3: memory improvements (SAMURAI, DAM4SAM, SAM2Long) evaluated per failure mode and on MOSEv2.
3. Phase 4: multi-object tracking (SAM2MOT vs. ByteTrack on DanceTrack); SAM 3.1 text prompts.
4. Phase 5: production hardening: async jobs, GPU image, streaming loader, artifact signing, auth.

8. Reproducibility

```
make setup setup-vos        # venv, API deps, CPU PyTorch, official SAM 2 @ 2b90b9f
make weights samples synth  # checkpoint, DAVIS car-shadow + SA-V example + demo clips, synthetic
videos
make test                   # fast suite (SQLite + fake segmenter)
make test-slow              # real SAM 2 inference
make benchmark              # configs/phase1.yaml -> results/phase1
python -m src.vos.cli profile   # encoder vs per-object cost on this machine
make report                 # plots, demo videos, portfolio cover
```

Item	Value
Python / PyTorch	3.13.16 / 2.14.1+cu130
Platform	Linux-6.18.44-fc-v114-x86_64-with-glibc2.39
Threads / device	2 / cpu
Checkpoint	sam2.1_t.pt (torch.float16)
Checkpoint SHA-256	3c1e81ca9b037dd39d70a014ddb9a813d6c4c4e12555420db7eaff31689bd4e3
Source of weights	github.com/ultralytics/assets v8.3.0 mirror of Meta SAM 2.1 hiera-tiny (fp16)
SAM 2 code	facebookresearch/sam2 @ 2b90b9f5 (Dec 2024)
Benchmark wall time	≈ 81 min (main run) + ≈ 8 min (appended stride sweep), 2 vCPU

Appendix A – Upwork portfolio entry

Field	Text
Project title (≤ 70)	AI Video Object Tracking & Segmentation with SAM 2 (PyTorch, FastAPI) [69 chars]
Your role (≤ 100)	Computer Vision / ML Engineer: SAM 2 pipeline, J&F evaluation, FastAPI serving, monitoring [90 chars]
Project description (≤ 600)	Goal: prove whether AI can keep a pixel-accurate mask on objects through a whole video (rotoscoping, privacy blur, tracking). Solution: SAM 2.1 pipeline with click/box prompts, DAVIS-standard J&F evaluation, stress tests (occlusion, exit/re-entry, look-alikes, fast motion) and label-free quality alerts, served via a FastAPI model registry with Prometheus/Grafana. Impact: J&F 0.987 from one click, instant recovery after a 12-frame occlusion, tracking failures flagged without labels (AUROC 0.97). 122 tests, Docker, CI. [522 chars]
Skills and deliverables (5)	Computer Vision · Object Tracking · Image Segmentation · PyTorch · Deep Learning

Content blocks to add

- **Image** (first = thumbnail): `portfolio/01_cover.jpg`: four tracked clips with headline metrics.
- **Video:** `portfolio/02_demo.mp4`: real two-person tracking, DAVIS, occlusion recovery, the exit failure, the distractor.
- **Image:** `portfolio/03_results.png`: J&F per scenario + per-frame J (persistence and recovery).
- **Image:** `portfolio/04_architecture.png`: registry → loader → SAM 2 → monitoring.
- **Text:** the deliverables list (rotoscoping CLI, video API, evaluation harness, GT-free alerts, stress tests).
- **Link:** your GitHub repository URL.
- **File:** `portfolio/05_code_review.pdf`: this report.